

ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE
FAKULTA STAVEBNÍ, OBOR GEODÉZIE A KARTOGRAFIE
KATEDRA GEOMATIKY

název předmětu

GEOINFORMATIKA

*číslo
úlohy*
3

název úlohy

Grafové algoritmy

školní rok

2025/26

skupina

2

zpracovali

Králič Adam, Matějková Barbora

datum

2. 12. 2025

klasifikace

Technická zpráva

1 Zadání úlohy

Úkolem bylo implementovat Dijkstra algoritmus pro nalezení nejkratší cesty mezi dvěma uzly grafu. Vstupními daty je silniční síť doplněnou vybranými sídly.

Otestovat různé varianty volby ohodnocení hran grafu tak, aby nalezená cesta měla:

- nejkratší Eukleidovskou vzdálenost,
- nejmenší transportní čas (2 varianty).

Ve vybraném GIS konvertovat podkladová data pro grafové reprezentace představované neorientovaným grafem. Pro druhou variantu optimální cesty navrhnout vhodnou metriku, která zohledňuje rozdílnou dobu jízdy na různých typech komunikací dle jejich návrhové rychlosti a klikatosti. Využít např. poměr délky polylinie $P = \sum_{i=1}^n$ a vzdálenosti koncových bodů polylinie.

$$f = \frac{l(P)}{\|p_1 - p_n\|_2} \quad (1)$$

Každou z variant otestovat pro dvě různé cesty. Výsledky umístit do tabulky, vlastní cesty vizualizovat. Dosažené výsledky porovnat s vybraným navigačním SW.

Krok	Hodnocení
Dijkstra algoritmus.	20b
<i>Řešení úlohy pro grafy se záporným ohodnocením.</i>	+15b
<i>Nalezení nejkratších cest mezi všemi dvojicemi uzlů.</i>	+15b
Nalezení minimální kostry Kruskal.	+15b
Nalezení minimální kostry Prime.	+15b
<i>Využití heuristiky Weighted Union</i>	+5b
<i>Využití heuristiky Path Compression</i>	+5b
Max celkem:	90b

Čas zpracování: 2 týdny.

2 Grafové algoritmy

Vysvětlení použitých grafových algoritmů je silně inspirováno z knihy Grafy a jejich aplikace od Jiřího Demela [1].

2.1 Nejkratší cesta

Jedná se o jednu z nejčastěji aplikovaných úloh teorie grafů, při které se hledá orientovaná cesta s nejmenším nákladem

- (a) z daného výchozího vrcholu do daného cílového vrcholu,
- (b) z daného výchozího vrcholu do každého vrcholu grafu,
- (c) z každého vrcholu do daného cílového vrcholu,
- (d) mezi všemi uspořádanými dvojicemi vrcholů.

Možnost a) se například řeší pomocí možností b), c) nebo d), jelikož efektivnější způsob není znám. Možnost c) lze převést na možnost b) při změně orientace cesty a metodu d) lze vytvořit metodou b) provedenou pro každý bod grafu. Nejkratší cestu lze realizovat mnoha způsoby, například modifikací prohledávání do šířky nebo do hloubky s postupným ukládáním vzdáleností. V této práci byly zvoleny nejefektivnější algoritmy, které v současnosti existují. V základu byl použit Dijkstra, který se používá pro grafy bez hran o záporných cenách. Dále byl implementován Bellman-Ford, který se používá pro grafy se záporným ohodnocením.

Pro velké soubory dat se často graf nejprve prohledá, zda obsahuje záporné hrany, a v návaznosti na to se zvolí použitý algoritmus. Podobný přístup se v grafových úlohách využívá často, jelikož pro různé typy grafů se mohou optimální způsoby řešení lišit.

2.2 Minimální kostra

Kostra je faktor grafu¹, jenž je souvislý². Koster může graf obsahovat početně mnoho³ a nejméně jedna je minimální, což znamená, že cena všech hran v této kostře je nejmenší možná.

Dnes se používají zejména dva algoritmy.

- Kruskalův – složitost $\mathcal{O}(m \log n)$, vhodnější pro grafy o větším počtu vrcholů (řidší grafy)
- Primův – složitost $\mathcal{O}(n \log m)$, vhodnější pro grafy o větším počtu hran (úplné grafy)

¹obsahuje všechny vrcholy původního grafu a některé původní hrany, po kterých je stále možné projít všechny vrcholy grafu

²všechny body grafu jsou spojeny hranou, neexistuje jiný izolovaný vrchol či množina vrcholů

³pokud je graf konečný

3 Použitá data

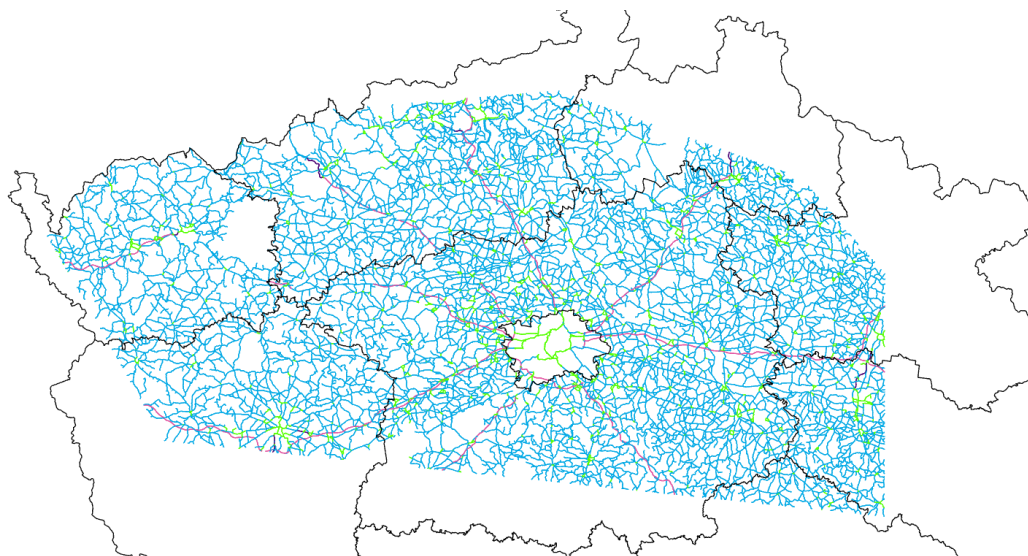
3.1 Varianty hodnocení grafu

Pro různé varianty ocenění hran grafu byly v této úloze použity možnosti doporučené zadáním – Eukleidovská vzdálenost definována délkami linií, transportní čas specifikovaný maximální povolenou rychlostí vynásobený délkou segmentu a transportní čas vynásobený mírou klikatosti definovanou rov. 1.

3.2 Získání dat

Prvotní zvolenou cestou byla cesta mezi městy Sokolov a Přelouč⁴, čemuž byla uzpůsobena použitá data.

Z různých datových zdrojů byla vybrána vrstva **Silnice, Dálnice**⁵ z databáze ZABAGED, jejíž objekty v atributové tabulce nesou informaci o typu silnice (dálnice, silnice II. třídy, ...). Byla nahrána do programu ArcGIS Pro a dále byla pro rychlejší výpočty oříznuta na plochu zájmu (obr. 1) specifikovanou primárně zájmovými městy.



Obrázek 1: Silnice v ploše zájmu; zelená = 50 km/h, modrá = 90 km/h, tmavě fialová = 110 km/h, červená = 130 km/h

Pro omezení rychlosti na 50 km/h v městských oblastech byly využity vrstvy sídel z databáze ArcČR500, konkrétně polygonová vrstva sídel obsahující okresní a větší města a zároveň bodová vrstva představující pouze definiční body menších obcí.

Kolem definičních bodů obcí byl vytvořen buffer o velikosti 1,2 km a polygony sídel byly také rozšířeny metodou buffer o 500 m. Následně byly tyto vrstvy spojeny do jedné a metodou **within** byly vybrány silnice, které se celou svou geometrií vyskytovaly v tomto polygonu, a byla jim přidělena rychlost 50 km/h.

Na závěr byl vytvořen příkaz pro přiřazení rychlosti zbylým třídám silnic.

⁴domácími městy zpracovatelů této úlohy

⁵https://geoportal.cuzk.cz/Dokumenty/ZABAGED_katalog/CS/2_Komunikace/ft_ap001.html


```

def give_speed(typ, max_speed):
    speed = 90                                # default speed to 90 km/h
    if max_speed = 50:                         # if speed was already set to 50 km/h
        speed = 50                             # returns it back
    highway = ["D1", "D2", "D1p", "D2p"]      # codes for highway
    if typ in highway:                         # if highway
        speed = 130                             # assigns speed 130 km/h
    elif typ in ["M", "Mp"]:                  # if motorway
        speed = 110                             # assigns speed 110 km/h
    return speed                               # returns new max speed

road.max_speed = give_speed(road.typsil_k, road.max_speed)

```

Tento zápis přidělí dálnicím a silnicím pro motorová vozidla odpovídající rychlost, silnicím ve městech zůstane rychlost 50 km/h a zbylým silnicím je přidělena rychlost 90 km/h. Takto upravená data neodráží přesný stav silnic a jejich rychlostní přechody, ale pro přibližnou velkoplošnou analýzu této úlohy byla dostatečná.

3.3 Použití Spatial SQL

Vrstva silnic byla následně vyexportována ve formátu **Shapefile** a byla nahrána do programu QGIS. Tento program byl zvolen pro umožnění použití **Spatial SQL**, které dovede efektivně extrahovat definiční body linie (počáteční a koncový) s ohodnocujícím parametrem.

Každý příkaz vytvořil novou tabulku, jejíž hodnoty byly uloženy do textových souborů, které byly použity ve výpočetním skriptu v programu Python. Každý řádek tohoto souboru má podobu (Start-X-Coordinate, Start-Y-Coordinate, End-X-Coordinate, End-Y-Coordinate, weight)

Dotazy byly prováděny pomocí QGIS DB manager a měly následující podobu.

3.3.1 Eukleidovská vzdálenost

```

SELECT
    Round(X(StartPoint(CastToLineString(GeometryN(a.geometry, 1)))),3) AS start_x,
    Round(Y(StartPoint(CastToLineString(GeometryN(a.geometry, 1)))),3) AS start_y,
    Round(X(EndPoint(CastToLineString(GeometryN(
        a.geometry, NumGeometries(a.geometry))))) ,3) AS end_x,
    Round(Y(EndPoint(CastToLineString(GeometryN(
        a.geometry, NumGeometries(a.geometry))))) ,3) AS end_y,
    Round(Shape_Leng,3) as delka
FROM silnice_final as a;

```

3.3.2 Eukleidovská vzdálenost + rychlost

```

SELECT
    Round(X(StartPoint(CastToLineString(GeometryN(a.geometry, 1)))),3) AS start_x,
    Round(Y(StartPoint(CastToLineString(GeometryN(a.geometry, 1)))),3) AS start_y,
    Round(X(EndPoint(CastToLineString(GeometryN(
        a.geometry, NumGeometries(a.geometry))))) ,3) AS end_x,

```

```

Round(Y(EndPoint(CastToLineString(GeometryN(
a.geometry, NumGeometries(a.geometry))))) ,3) AS end_y,
Round(Shape_Leng/max_speed,4) as delka_rychlost
FROM silnice_final as a;

```

3.3.3 Eukleidovská vzdálenost + rychlost + klikatost

Parametr klikatosti byl vypočten přímo pomocí Spatial SQL a vnořené operace SELECT.

```

SELECT
    q.start_x,
    q.start_y,
    q.end_x,
    q.end_y,
    ROUND(
        q.Shape_Leng / (SQRT((q.start_x - q.end_x)*(q.start_x - q.end_x) +
            (q.start_y - q.end_y)*(q.start_y - q.end_y)) / q.Shape_Leng)
        / q.max_speed
    , 4) AS delka_rychlost_klikatost
FROM (
    SELECT
        a.Shape_Leng,
        a.max_speed,
        ROUND(X(StartPoint(CastToLineString(GeometryN(a.geometry, 1))))) , 3) AS start_x,
        ROUND(Y(StartPoint(CastToLineString(GeometryN(a.geometry, 1))))) , 3) AS start_y,
        ROUND(X(EndPoint(CastToLineString(GeometryN(
            a.geometry, NumGeometries(a.geometry))))) , 3) AS end_x,
        ROUND(Y(EndPoint(CastToLineString(GeometryN(
            a.geometry, NumGeometries(a.geometry))))) , 3) AS end_y
    FROM silnice_final AS a
) AS q;

```

4 Vytvořené algoritmy

Pro tuto práci byly vytvořeny algoritmy, které lze rozdělit do 3 kategorií.

- **Algoritmy pro zpracování dat** (příloha 2) – umožňují převod hodnot z textových souborů do proměnných obsahující informace o grafech či jiných pomocných strukturách.
- **Grafové algoritmy** (příloha 3) – Jedná se o implementace algoritmů z teorie grafů a vytvoření potřebných datových struktur pro následné vizualizace.
- **Vykreslovací algoritmy** (příloha 4) – Algoritmy vytvořené pro automatizované vykreslení výstupů z grafových algoritmů.

4.1 Algoritmy pro zpracování dat

4.1.1 Vytvoření struktury grafu

Pro zjednodušení práce na úloze bylo několik těchto algoritmů poskytnuto již implementovaných se zadáním. Jedná se o funkce `loadEdges()`, `pointsToIDs()` a `edgesToGraph()`. Výsledný algoritmus `edgesToGraph()` vrací strukturu grafu, která obsahuje pro každý bod jejich výstupní uzly a patřičné ocenění hran k nim vedoucím.

```
def loadEdges(file_name):                                # convert list of lines to the graph
    PS = []                                              # start points
    PE = []                                              # end points
    W = []
    with open(file_name) as f:                          # opens input file with roads
        for line in f:
            x1, y1, x2, y2, w = line.split()           # splits the input
            PS.append((float(x1), float(y1)))           # append start point
            PE.append((float(x2), float(y2)))           # append end point
            W.append(float(w))                          # append weight
    return PS, PE, W

def pointsToIDs(P):                                     # creates a map: key = coordinates, value = id
    D = {}
    for i in range(len(P)):
        D[(P[i][0], P[i][1])] = i                     # for group of coordinates assigns a new ID
    return D

def edgesToGraph(D, PS, PE, W):                         # converts edges to undirected graph
    G = defaultdict(dict)
    for i in range(len(PS)):
        G[D[PS[i]]][D[PE[i]]] = W[i]
        G[D[PE[i]]][D[PS[i]]] = W[i]
    return G
```

Pro tvorbu minimální kostry byly vytvořeny další grafové struktury, přesněji seznam vrcholů a seznam hran, kde jsou hrany uloženy v podobě: (počáteční uzel, koncový uzel, cena).

```

def graph_to_nodes(G):          # takes main graph structure and extracts node IDs
    V = []
    for node, _ in G.items():
        V.append(node)         # appends node ID
    return V

def graph_to_edges(G):          # used for Kruskal
    E = set()                   # set: doesn't need to be organized, only unique
    n = len(G)
    for u in range(n):
        for v, wuv in G[u].items():
            E.add((u, v, wuv)) # adds new edge
    return E

```

Celý tento proces byl „zabaleno“ do výsledné funkce, která z textové souboru vytvoří proměnné potřebné pro použité grafové algoritmy.

```

def file_to_graph(file):
    PS, PE, W = loadEdges(file)      # process edges
    PSE = PS + PE                    # merge lists and remove unique points
    PSE=unique(PSE,axis=0).tolist()
    PSE.insert(0, [1000000, 1000000])
    D = pointsToIDs(PSE)              # edges to graph
    G = edgesToGraph(D, PS, PE, W)
    V = graph_to_nodes(G)             # nodes from graph
    E = graph_to_edges(G)             # graph to edges
    return D, G, V, E                # returns everything necessary

```

4.1.2 Nalezení nejbližšího města

Aby bylo možné volit uzly pro rekonstrukci nejkratších cest intuitivněji než jako pouhá čísla ze seznamu, byl vytvořen algoritmus pro nalezení nejbližšího definičního bodu silnice k definičnímu bodu obce sestávající se ze dvou částí.

- **Načtení souboru (`obce_file_to_dict_list()`)** – tento algoritmus, přijímá na vstupu soubor, který obsahuje v každém řádku název obce a souřadnice definičního bodu a vytváří z nich nové proměnné – seznam názvů obcí a slovník ve formátu město: (X-Coordinate, Y-Coordinate).

```

def obce_file_to_dict_list(filename): # creates a cities dictionary
    # with value = city, key = (X, Y) and a list of cities
    obce = dict()
    obce_names = list()
    with open(filename, "r", encoding="utf-8") as f: # opens and reads the file
        for line in f: # process each line
            parts = line.strip().split("\t")         # splits by blanc spaces

```

```

        name = parts[0]                                # extracts the parts list
        x = float(parts[1])
        y = float(parts[2])
        obce[name] = (x, y)                             # adds new key:value
        obce_names.append(name)                         # appends city name
    return obce, obce_names

```

- **Vyhledání nejbližšího bodu silnice** (`city_to_node(city_name, D, obce)`) – funkce zkontroluje, zda se název města nachází v seznamu měst z předchozího kroku a následně prohledává všechny body silniční sítě a vrací ten, který je nejbližší definičnímu bodu obce.

```

def city_to_node(city_name, D, obce):                  # city name to road node ID
    try:                                                # tries if the city name exist
        city = obce[city_name]
    except KeyError:
        raise Exception(f"Město {city_name} není v databázi")
    shortest = 1000000                                # starting shortest distance
    shortest_key = 0
    for value, key in D.items():                       # for node in road
        distance = sqrt(pow(city[0]-value[0],2) + pow(city[1]-value[1],2)) # pythagoras
        if distance < shortest:                       # if smaller than previous shortest
            shortest = distance                       # overwrites shortest distance
            shortest_key = key                        # overwrites closest road node
    return shortest_key

```

4.2 Grafové algoritmy

V této práci byly realizovány všechny povinné a bonusové úlohy, což zahrnuje algoritmy na vyhledání nejkratší cesty mezi dvojicí uzlů pomocí Dijkstry pro hrany s kladným ohodnocením a pomocí Bellman-Fordova algoritmu pro grafy s ohodnocením záporným. Dále byl implementován algoritmus pro nalezení nejkratších cest mezi všemi dvojicemi uzlů a algoritmy pro tvorbu minimální kostry (Kruskal, Prime).

4.2.1 Nejkratší cesta – Dijkstra

Funkce `dijkstra(G, start, end)` implementuje Dijkstrův algoritmus pro hledání nejkratší cesty v orientovaném grafu s nezáporným ohodnocením hran. Hlavní dvě proměnné jsou seznam předchůdců (`pred`), který na indexu požadovaného vrcholu obsahuje index vrcholu svého předchůdce, a seznam vzdáleností (`dist`) pro ukládání dosud známé nejkratší vzdálenosti ke každému vrcholu.

Ve výchozím stavu se vzdálenost počátečního vrcholu nastaví na nulu, zatímco vzdálenost ostatních vrcholů na nekonečno. Dále se založí prioritní fronta PQ, do které se vkládají dvojice (vzdálenost, vrchol). Tato fronta vždy vrací vrchol s nejmenší aktuální vzdáleností.

Algoritmus pak opakovaně odebírá následující vrchol v prioritní frontě a pro každý jeho sousední vrchol zkusí provést relaxaci⁶ a probíhá dokud není prioritní fronta prázdná.

```
def dijkstra(G, start, end):
    n = len(G)
    pred = [-1]*(n+1)
    dist = [math.inf]*(n+1)
    PQ = queue.PriorityQueue()
    PQ.put((0, start))
    dist[start] = 0
    while not PQ.empty():
        du, u = PQ.get()
        for v, wuv in G[u].items():
            if dist[v] > dist[u] + wuv:
                dist[v] = dist[u] + wuv
                pred[v] = u
                PQ.put((dist[v], v))
    return pred, dist[end]
```

Dijkstra algorithmus
node count
predecessors
distances
creates priority queue
add start vertex
repeats until PQ is empty
gets point with lowest du
browse all adjacent nodes
relax
updates dv
stores predecessor
add v to priority queue
returns list of predecessors and dist.

4.2.2 Nejkratší cesta – Bellman-Ford

Tento algoritmus, na rozdíl od Dijkstra, umí nalézt nejkratší cesty i se záporným ohodnocením hran⁷.

Implementace je podobná, liší se pouze tím, že nevyužívá prioritní frontu. Místo ní jsou použity dva vnořené cykly, ve kterých je n-krát pro každý vrchol provedena relaxace. Pokud při průběhu vnějšího cyklu nedojde ke zlepšení, algoritmus je předčasně ukončen pro zlepšení efektivity.

```
def bellman_ford(G, start, end):
    n = len(G)
    pred = [-1]*(n+1)
    dist = [math.inf]*(n+1)
    dist[start] = 0
    for i in range(n+1):
        improved = False
        for u in range(n):
            for v, wuv in G[u].items():
                if dist[u] != math.inf and dist[v] > dist[u] + wuv:
                    dist[v] = dist[u] + wuv
                    pred[v] = u
                    improved = True
        if not improved:
            break
    if i == n:
        return None
    return pred, dist[end]
```

implementation of Bellman-Ford
node count
predecessors
distances
relax edges n times
for each node
brose all adjacent nodes
updates dv
stores predecessor
improved
if no path is better, algorithm can stop sooner
if negative cycle exists -> returns none
returns list of predecessors and dist.

⁶zkouška trojúhelníkové nerovnosti: pokud je nově vypočtená vzdálenost ($\text{dist}[u] + wuv$) kratší než dosavadní vzdálenost $\text{dist}[v]$, pak se tato vzdálenost aktualizuje, uloží se předchůdce a vrchol se znovu vloží do prioritní fronty.

⁷stále ale nesmí existovat záporný cyklus, jinak by tato cesta neexistovala, limitně by se blížila k $-\infty$

4.2.3 Nejkratší cesta – volba algoritmu a nejkratší cesta mezi všemi dvojicemi uzlů

Pro zvolení použitého algoritmu k nalezení nejkratší cesty byla vytvořena funkce `shortest_path(G, start = None)`. Prohledá graf získaný na vstupu a na základě existence záporných hran zvolí, zda je možné použít Dijkstrův algoritmus, nebo daný graf vyžaduje Bellman-Fordův.

Dále pokud uživatel nedefinuje počáteční uzel, bude proveden výpočet nejkratší cesty mezi všemi dvojicemi uzlů. Tento proces je realizován Dijkstrou pro všechny body v grafu. Výpočet trvá relativně dlouho, pro naši analýzu o 12 095 uzlech trvá na průměrném zařízení kolem 8 minut. Každý seznam předchůdců je uložen do nového seznamu.

```
def shortest_path(G, start = None, end = 1):
    n = len(G)                                # node count
    if start is not None:                     # if start node is defined
        negative_edges = False
        for u in range(n):                   # finds negative edges
            for v, wuv in G[u].items():
                if wuv < 0:
                    negative_edges = True
        if negative_edges is True:            # if negative edges -> bellman-ford
            pred, dist = bellman_ford(G, start, end)
        else:                                 # if only positive edges -> dijkstra
            pred, dist = dijkstra(G, start, end)
        return pred, dist
    else:                                     # if no point is defined as start point
        # calculates shortest paths (dijkstra) from every point to every point
        pred = [None]*(n+1)                  # new list for lists of predecessors
        for u in range(1, n):
            if u % 100 == 0:
                pred[u], dist = dijkstra(G, u, end)
        return pred, dist
```

4.2.4 Minimální kostra – Kruskal

Pro tento algoritmus byly vytvořeny 3 pomocné funkce.

- `make_set()` – vytvoří nový strom z jednoho vrcholu, používá se na začátku Kruskalova algoritmu pro každý vrchol.

```
def make_set(u, pred, rank):    # makes new set
    pred[u] = u                 # sets itself as predecessor
    rank[u] = 0                 # rank of tree = 0
```

- `find()` – nalezne kořen stromu se zadaným vrcholem `u`. Pro tuto metodu byla implementována bonusová heuristika `Path Compression`, kde je možno si zvolit mezi

1. úplnou kompresí (`path_compression = "full"`) – předchůdce požadovaného vrcholu se nastaví kořen stromu, ve kterém se nachází.

2. poloviční komprese (`path_compression = "half"`) – předchůdce požadovaného vrcholu se nastaví předchůdce svého aktuálního předchůdce → zkrácení cesty hledání kořene na polovinu.
3. žádná komprese (`path_compression = None`) – seznam předchůdců se nijak nemění, metoda `find()` musí projít všechny předchůdce, dokud se nedostane ke kořenu stromu, ve kterém se vrchol nachází.

```
def find(pred, u, path_compression = None): # find root
    if path_compression == "half":         ## half compression
        while pred[u] != u:
            pred[u] = pred[pred[u]]        # moves to grandparent
            u = pred[u]                    # predecessor is grandparent
        return u
    elif path_compression == "full":        ## full compression
        while pred[u] != u:                # finds root
            u = pred[u]
        root = u
        while u != root:
            u_pred = pred[u]               # stores predecessor
            pred[u] = root                 # changes predecessor to root
            u = u_pred                     # goes to parent
        return u
    else:                                   ## no path compression
        while pred[u] != u:
            u = pred[u]
        return u
```

- `union()` – spojí dva stromy k sobě, neboli kořen stromu s menším počtem prvků se nastaví jako kořen většího stromu. V této funkci byla také implementována bonusová heuristika **Weighted Union**, která zajišťuje, aby se kořen menšího stromu změnil na kořen stromu většího.

```
def union(pred, u, v, rank, path_compression=None):
    # finds roots of start and end nodes
    root_u = find(pred, u, path_compression=path_compression)
    root_v = find(pred, v, path_compression=path_compression)
    if root_u != root_v: # if roots are different -> joined together
        if rank[root_u] > rank[root_v]: # weighted union
            pred[root_v] = root_u
        elif rank[root_v] > rank[root_u]: # weighted union
            pred[root_u] = root_v
    else: # if roots are same size, v is joined to u
        pred[root_u] = root_v
        rank[root_v] = root_v + 1
```

Na vstupu funkce je množina vrcholů V a množina hran E . Podobně jako u nejkratších cest obsahuje tento algoritmus stav o předchůdcích jednotlivých vrcholů a nově se uvádí **rank** stromu,

informace o tom, jak je strom dlouhý od kořene k nejvzdálenějšímu listu, a také množina T , jež bude obsahovat veškeré hrany tvořící minimální kostru. Na začátku je pro každý vrchol vytvořen samostatný strom pomocí funkce `make_set()`. Vytvoří se nový seznam hran, který obsahuje všechny původní hrany a je seřazený podle ceny hran.

Postupně se z této srovnané množiny tahají jednotlivé hrany a testuje se, zda počáteční a koncový bod mají stejný kořen. Pokud nemají, pomocí funkce `union()` se spojí do jednoho stromu a do množiny T je přidána nová platná hrana minimální kostry.

Zároveň se zde počítá i celková cena minimální kostry, která je před spuštěním skriptu rovna 0, a následně se k ní přičítají jednotlivé ceny hran, tvořící minimální kostru.

```
def boruvka_kruskal(V, E, p_c=None):
    T=[]                                # empty tree
    wt = 0                              # sum of weights of T
    pred = [-1] * (len(V) + 1)         # list of predecessors
    rank = [math.inf] * (len(E) + 1)   # ranks of the node
    for v in V:                         # makes set
        make_set(v, pred, rank)
    ES = sorted(E, key=lambda it:it[2]) # sorts edges by weight
    for e in ES:                        # processes all edges
        u, v, w = e                    # takes an edge
        if (find(pred, u, p_c) != find(pred, v, p_c)): # if different trees
            union(pred, u, v, rank)    # unites nodes
            T.append([u, v, w])        # adds edge to tree
            wt += w                    # adds weight to tree
    return wt, T
```

4.2.5 Minimální kostra – Prime

Pro implementaci tohoto algoritmu byla použita první jeho podoba bez přidání zrychlení (halda, atd.). Proto je samotný výpočet mnohem pomalejší, než při použití algoritmu Kruskal.

Implementovaná funkce má na vstupu množinu vrcholů V a grafovou strukturu G . Funkce definuje množinu T , která bude obsahovat hrany minimální kostry, ovšem je zde tvořena i nová množina vrcholů ve stromu `tree_nodes`, jenž je pro tento algoritmus zásadní.

Nejprve je přidán náhodný vrchol do množiny `tree_nodes` a vybere se nejlevnější hrana z něho vystupující. Ta se přidá do množiny hran stromu T a její koncový vrchol, se přidá do množiny vrcholů stromu `tree_nodes`.

Tento proces se následně opakuje, avšak s tím rozdílem, že se nejlevnější výstupní hrana vybírá z celé množiny vrcholů `tree_nodes` a zároveň musí vést do vrcholu mimo tuto množinu.

Jakmile jsou všechny dostupné vrcholy připojeny, nebo se nenajde žádná volná hrana (pokud by nebyl graf souvislý), funkce se ukončí a vrátí cenu minimální kostry s množinou hran T .

```
def jarnik_prime(V, G):
    T = []                                # empty tree
    wt = 0                              # sum of weights of T
    u = random.choice(V)                 # chooses random node
    tree_nodes = set([u])                # set of found nodes
```

```

n = len(G)                                # node count
while len(tree_nodes) != n:                # while there are nodes to add
    min_wuv = math.inf                     # minimal potential weight
    min_v = math.inf
    for node in tree_nodes:                # go through all nodes in tree
        for v, wuv in G[node].items():    # go through all adjacent nodes
            if v in tree_nodes:            # skip if adjacent node already in tree
                continue
            if wuv < min_wuv:               # if cheaper than cheapest edge
                min_wuv = wuv              # new cheapest edge
                min_v = v                  # adjacent node with cheapest edge
                u = node                   # store original node
    if min_wuv == math.inf:                 # if no other node was found
        break                             # breaks out of the cycle
    T.append([u, min_v, min_wuv])           # adds edge to the tree
    tree_nodes.add(min_v)                  # adds node to the tree
    wt += min_wuv # adds weight to tree    # adds weight to tree
return wt, T

```

4.2.6 Rekonstrukce cesty

Poslední algoritmus v této sekci slouží pro rekonstrukci nejkratší cesty ze seznamu předchůdců a je použit při její výsledné grafické vizualizaci.

Ve funkci je nový seznam, do kterého se postupně přidávají předchůdci pramenící z koncového uzlu, dokud se předchůdcem nestane počáteční uzel.

```

def reconstPath(pred, u, v):
    path = [v]                                # appending end node to path
    while v != u and v != -1:                 # path shortening
        v = pred[v]                           # updates predecessor
        path.append(v)                         # add to the list
    return path

```

Ve skriptu jsou vytvořeny ještě další funkce na prohledávání do šířky a do hloubky, leč nebyly v této úloze využity a nebudou zde popsány.

4.3 Vykreslovací algoritmy

Algoritmy vytvořené pro vykreslení grafových výstupů používají knihovnu Matplotlib. Skript obsahuje opět jednu funkci `plot_graph`, která vykreslí BFS nebo DFS strom, každopádně nebyla v úloze použita a nebude zde vypsána.

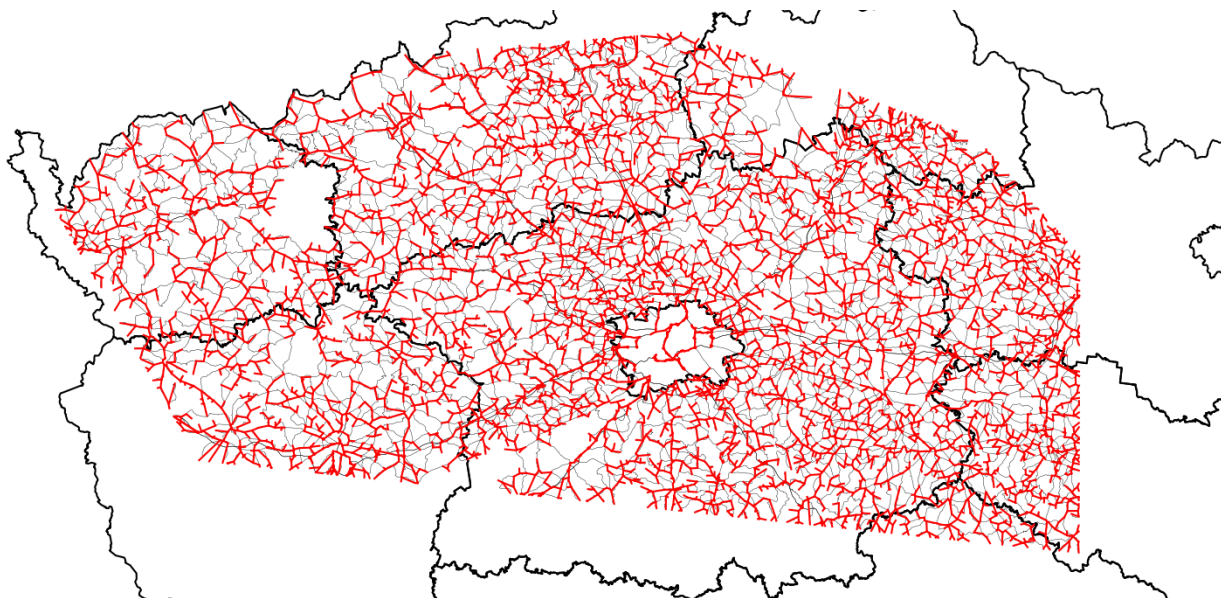
Funkce `plot_skeleton_tree(C, T, color)` na vstupu přebírá seznam souřadnic bodů `C` a množinu hran minimální kostry `T`. Každá hrana je vykreslena pomocí počátečního a koncového bodu linie. Pro data zohledňující jakožto cenu hrany pouze Eukleidovskou vzdálenost byla vykreslena minimální kostra (obr. 2).

```

def plot_skeleton_tree(C, T, color="r"): # plots a skeleton tree

```

```
# input: coordinates of all nodes, tree, color (voluntary)
for edge in T:
    node1_x, node1_y = C[edge[0]][0], C[edge[0]][1] # coordinates of start point
    node2_x, node2_y = C[edge[1]][0], C[edge[1]][1] # coordinates of end point
    plt.plot((node1_x, node2_x), (node1_y, node2_y), color=color)
```



Obrázek 2: Minimální kostra pro hrany o ceně Eukleidovské vzdálenosti

Funkce `plot_shp_polygons(sf, line_width, color="k")` vykreslí polygony nebo liniové vrstvy ve formátu ShapeFile. Pro přečtení geometrií jednotlivých objektů byla použita knihovna `Shapely`.

Algoritmus byl sepsán zejména kvůli špatnému vykreslení linií z jednoho polygonu do druhého a kvůli opakovanému použití v hlavním skriptu (příloha 1). Na vstupu umožňuje též volbu tloušťky linie a vykreslovací barvy.

```
def plot_shp_polygons(sf, line_width, color="k"): # plots shapefiles
    for shapeRec in sf.shapeRecords(): # takes one shape
        shp = shapeRec.shape
        parts = shp.parts.tolist() + [len(shp.points)] # extracts only polygon ring
        for i in range(len(parts) - 1): # plots each part separately
            start_i, end_i = parts[i], parts[i+1]
            x = [p[0] for p in shp.points[start_i:end_i]]
            y = [p[1] for p in shp.points[start_i:end_i]]
            plt.plot(x, y, "-", linewidth=line_width, color=color)
```

5 Vytvořené cesty

Jak již bylo zmíněno v sekci Použitá data, celkem byly v této úloze vypočteny nejkratší cesty pomocí tří metrik.

- Eukleidovská vzdálenost – znázorněna červenou barvou
- Eukleidovská vzdálenost + maximální povolená rychlost – znázorněna zelenou barvou
- Eukleidovská vzdálenost + maximální povolená rychlost + klikatost – znázorněna modrou barvou

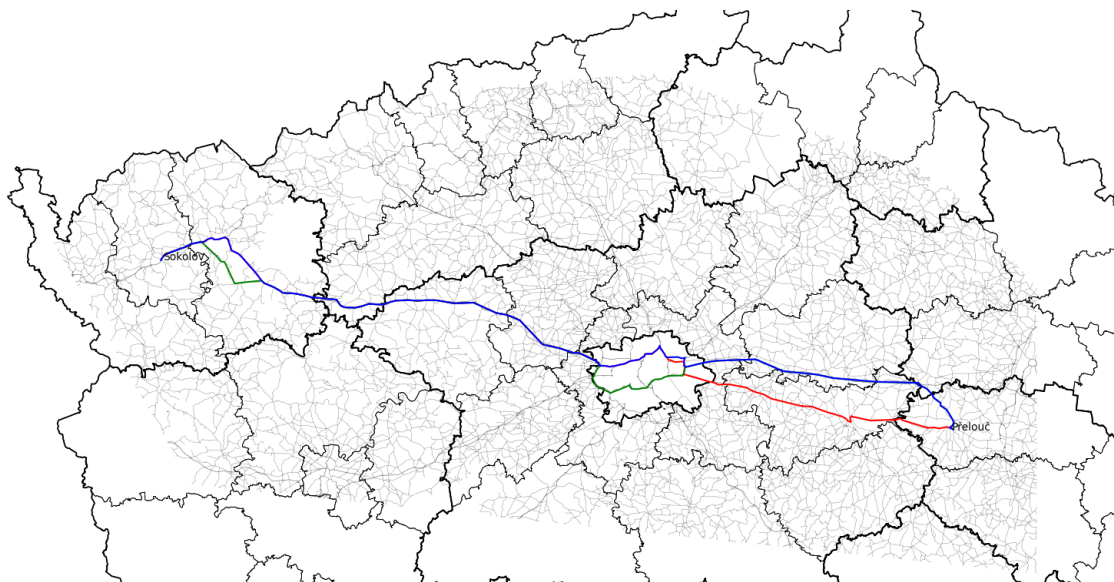
Všechny tyto metriky byly porovnány s navigačními aplikacemi Mapy.com, Google Maps a Apple Maps. Mapy.com umožňují volbu nejrychlejší cesty bez provozu a nejkratší cesty, což je pro porovnání s výsledky úlohy ideální. Avšak Google Maps a Apple maps tyto možnosti nenabízí, takže tyto varianty byly nahrazeny jinými nastaveními. Nejkratší vzdálenost byla nahrazena výsledkem pro trasu pěší, která následuje trasu silnic, a nejrychlejší cesta byla zvolena kolem desáté hodiny večerní, takže nebyla příliš ovlivněna provozem.

Celkem byly výsledky ilustrovány na třech trasách mezi městy

- Sokolov a Přelouč,
- Svojšín a Kanina,
- Úštěk a Uhlířské Janovice.

Sokolov → Přelouč				
–	vlastí skript	Mapy.com	Google Maps	Apple Maps
Eukleidovská vzdálenost	237,1 <i>km</i>	230,5 <i>km</i>	231,0 <i>km</i>	246,2 <i>km</i>
nejrychlejší	2 h 32 <i>min</i>	2 h 46 <i>min</i>	2 h 40 <i>min</i>	2 h 41 <i>min</i>
nejrychlejší + klikatost	2 h 39 <i>min</i>	–	–	–
Svojšín → Kanina				
–	vlastí skript	Mapy.com	Google Maps	Apple Maps
Eukleidovská vzdálenost	170,4 <i>km</i>	174,8 <i>km</i>	166,0 <i>km</i>	173,8 <i>km</i>
nejrychlejší	1 h 50 <i>min</i>	2 h 15 <i>min</i>	2 h 10 <i>min</i>	2 h 16 <i>min</i>
nejrychlejší + klikatost	1 h 58 <i>min</i>	–	–	–
Úštěk → Uhlířské Janovice				
–	vlastí skript	Mapy.com	Google Maps	Apple Maps
Eukleidovská vzdálenost	117,4 <i>km</i>	116,9 <i>km</i>	111,0 <i>km</i>	112,7 <i>km</i>
nejrychlejší	1 h 24 <i>min</i>	1 h 54 <i>min</i>	1 h 57 <i>min</i>	1 h 52 <i>min</i>
nejrychlejší + klikatost	1 h 29 <i>min</i>	–	–	–

Tabulka 1: Porovnání ceny nejkratších cest s různými navigačními aplikacemi



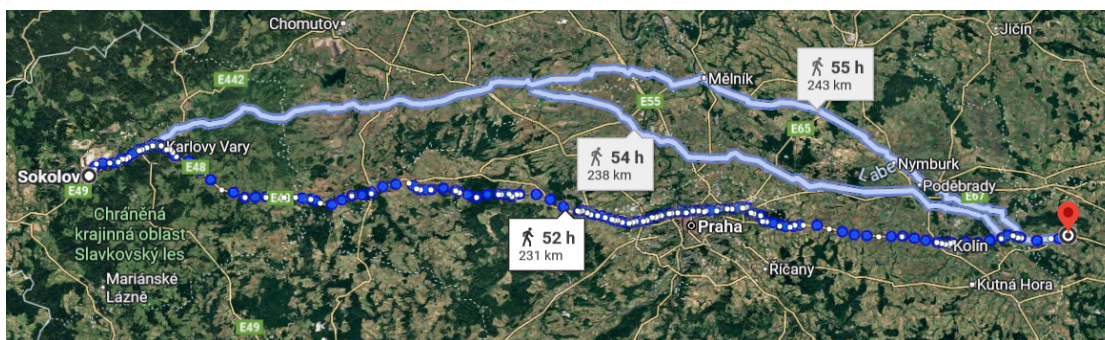
Obrázek 3: Vypočtená trasa mezi městy Sokolov a Písek



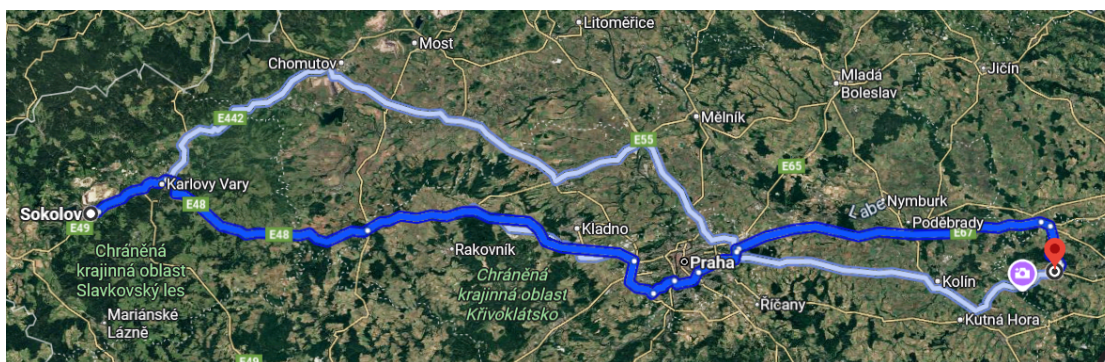
Obrázek 4: Nejkratší cesta ze stránky Mapy.com mezi městy Sokolov a Písek



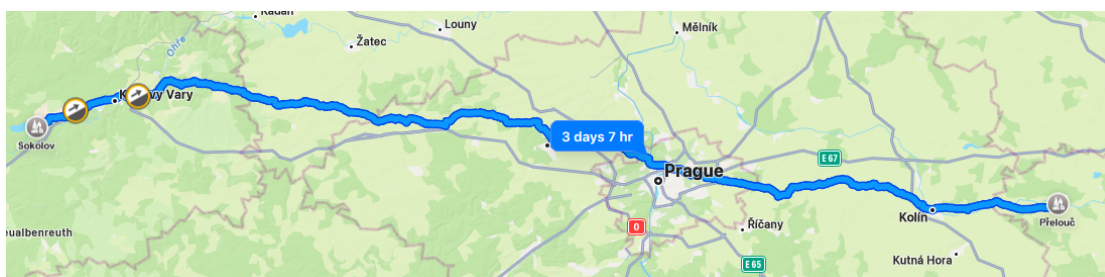
Obrázek 5: Nejrychlejší cesta ze stránky Mapy.com mezi městy Sokolov a Písek



Obrázek 6: Nejkratší cesta ze stránky Google Maps mezi městy Sokolov a Písek



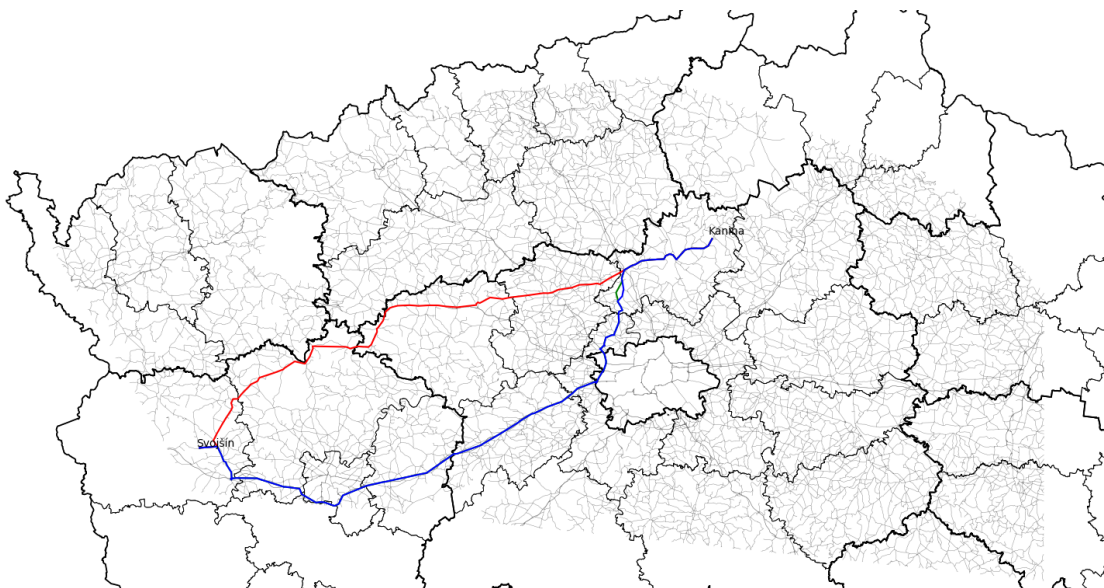
Obrázek 7: Nejrychlejší cesta ze stránky Google Maps mezi městy Sokolov a Písek



Obrázek 8: Nejkratší cesta ze stránky Apple Maps mezi městy Sokolov a Písek



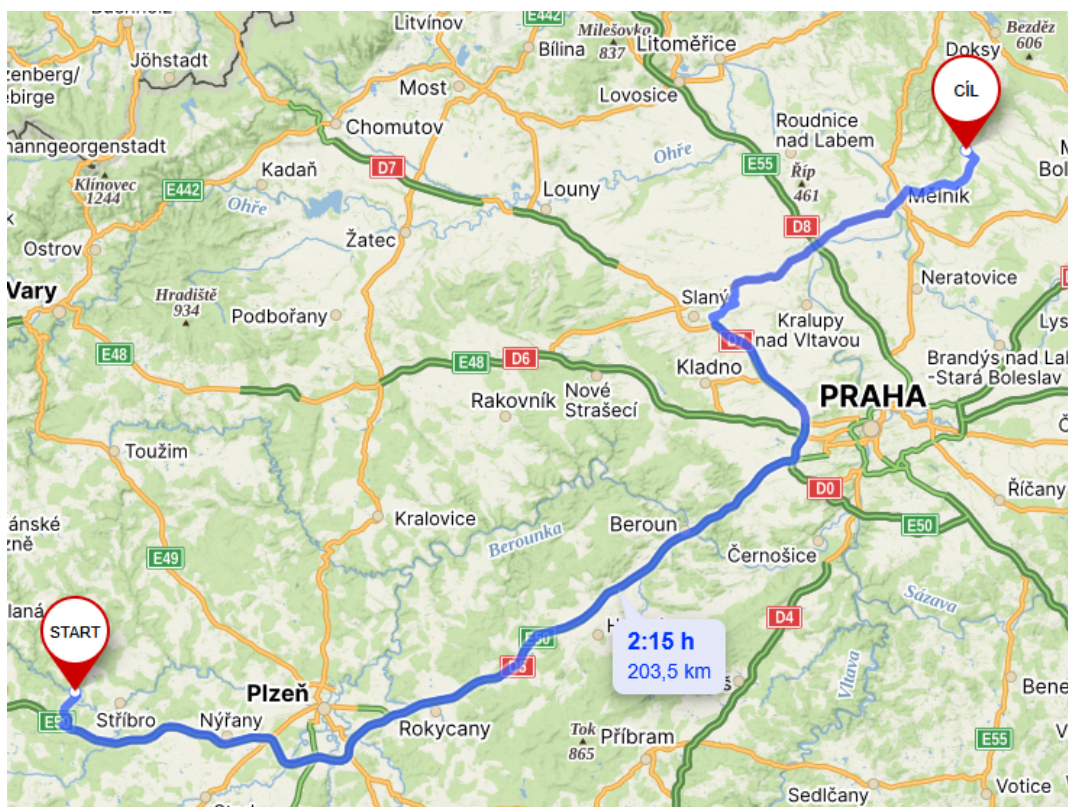
Obrázek 9: Nejrychlejší cesta ze stránky Apple Maps mezi městy Sokolov a Písek



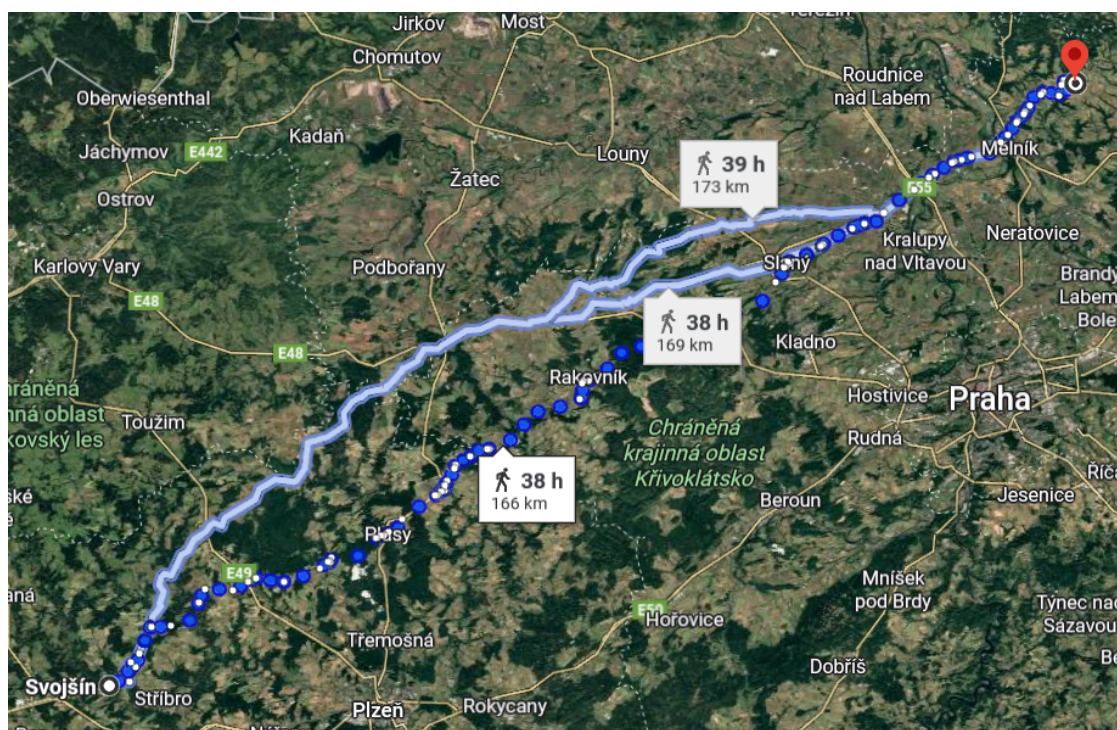
Obrázek 10: Vypočtená trasa mezi městy Svojsín a Kanina



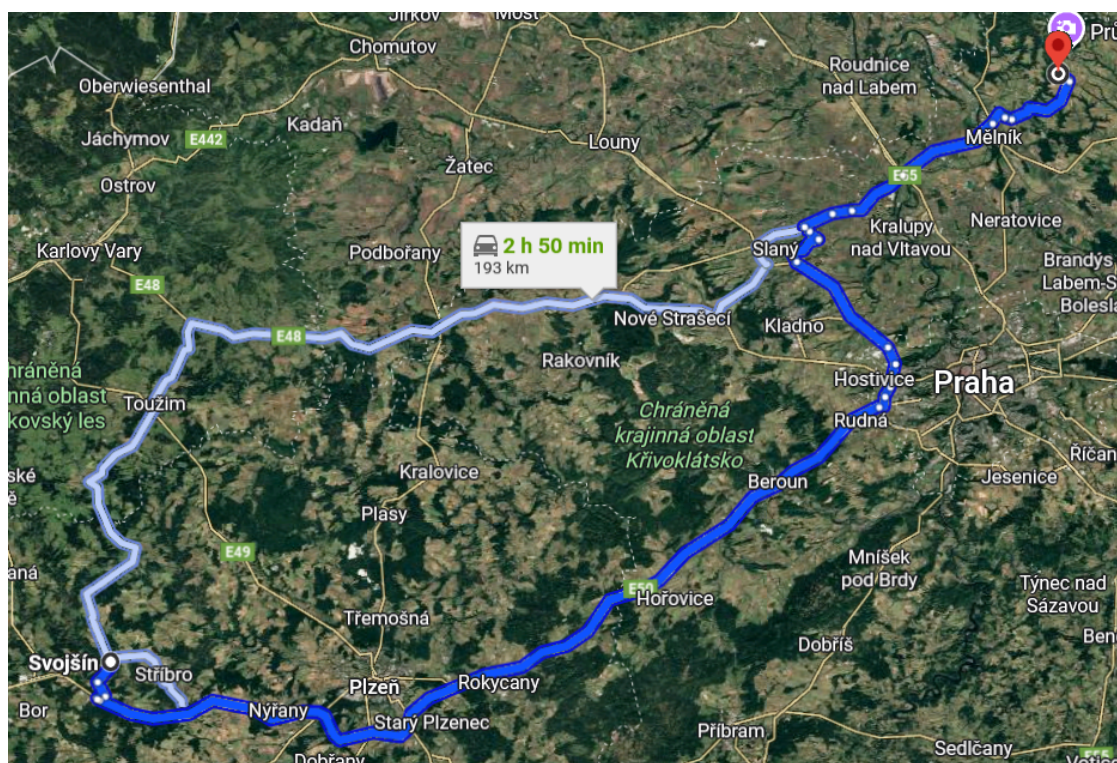
Obrázek 11: Nejkratší cesta ze stránky Mapy.com mezi městy Svojsín a Kanina



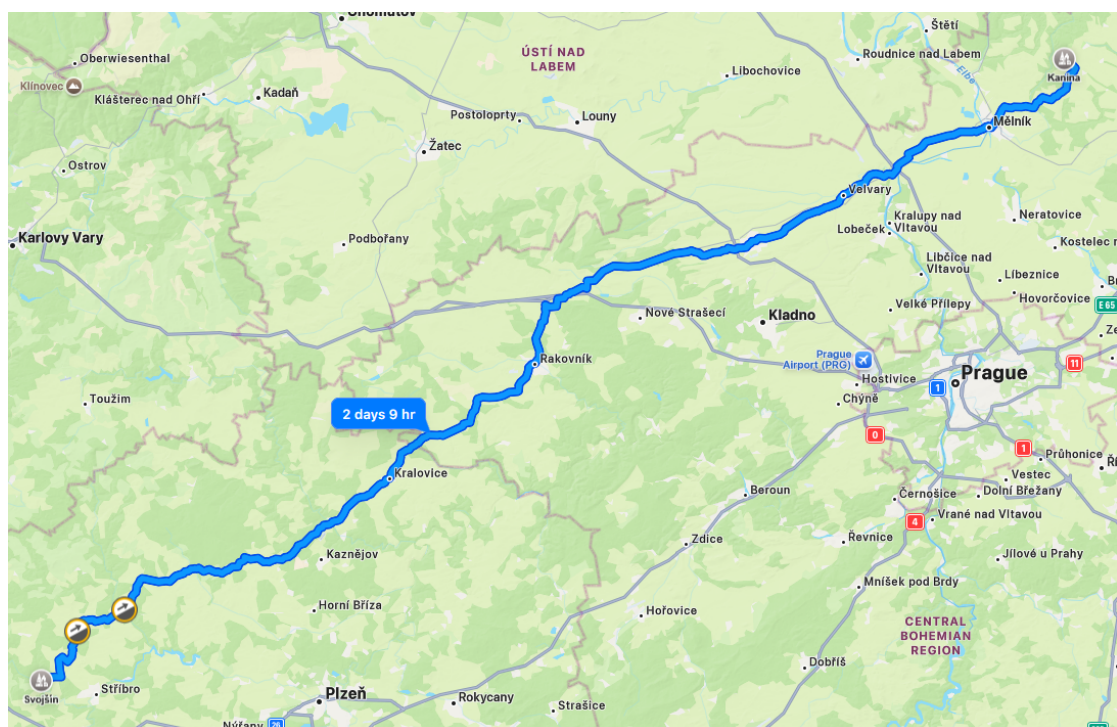
Obrázek 12: Nejrychlejší cesta ze stránky Mapy.com mezi městy Svojsín a Kanina



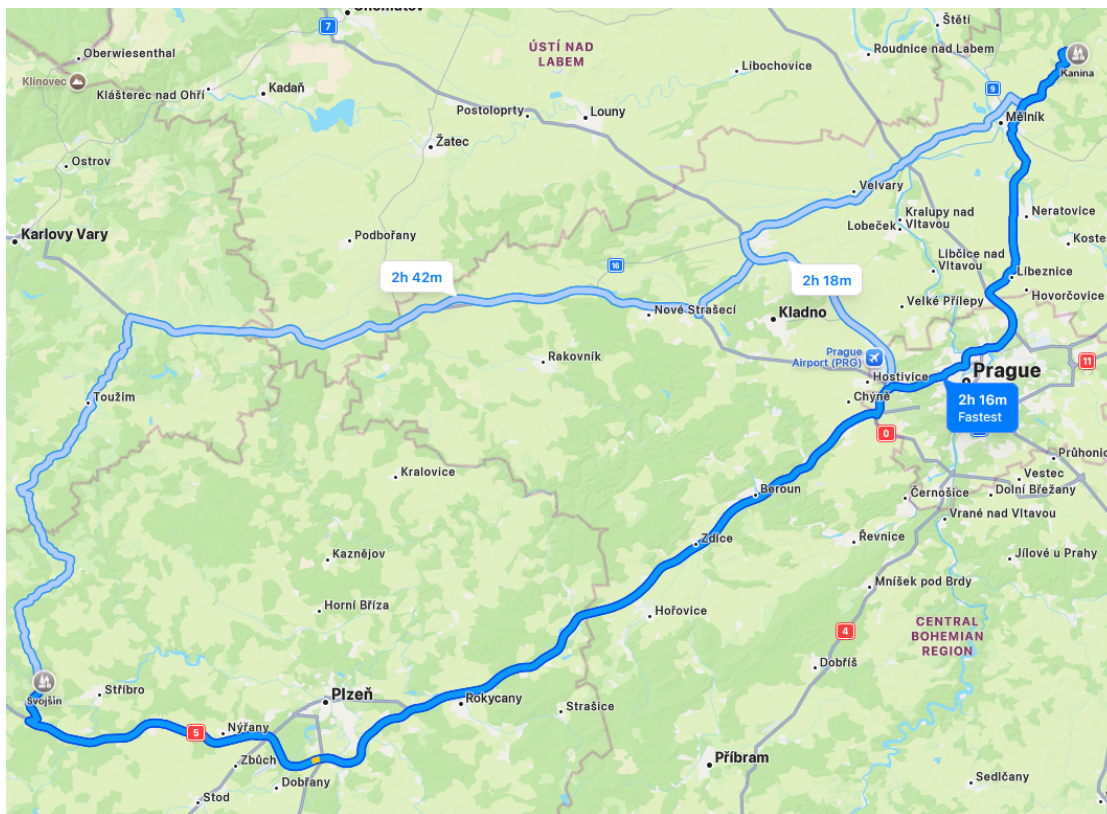
Obrázek 13: Nejkratší cesta ze stránky Google Maps mezi městy Svojsín a Kanina



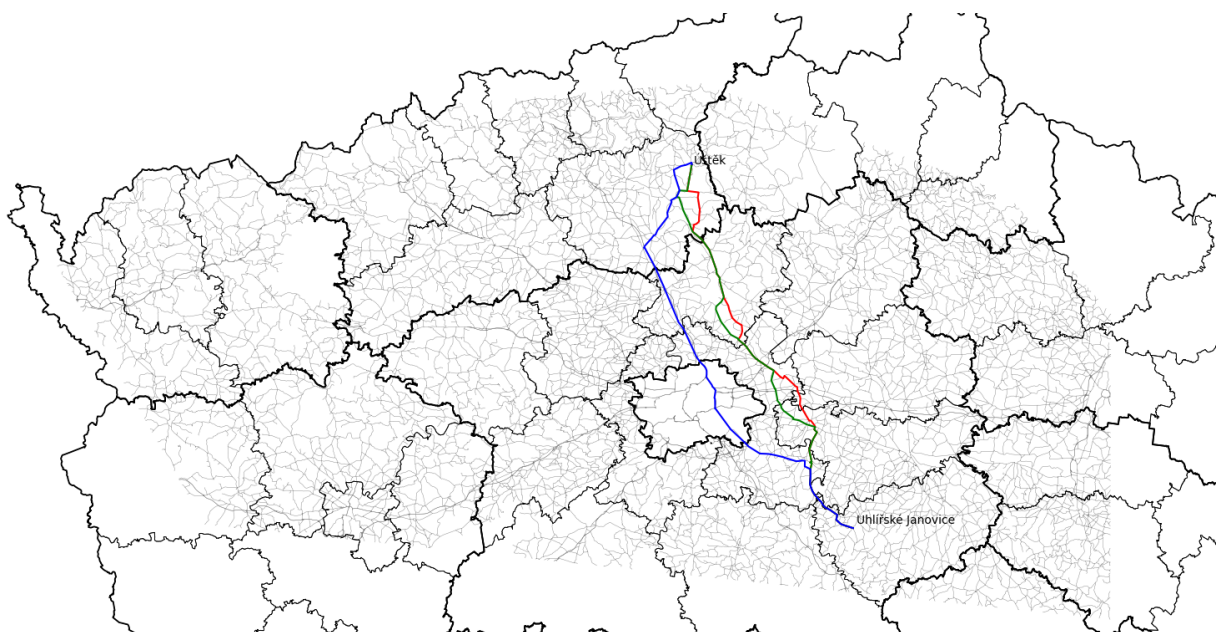
Obrázek 14: Nejrychlejší cesta ze stránky Google Maps mezi městy Svojšín a Kanina



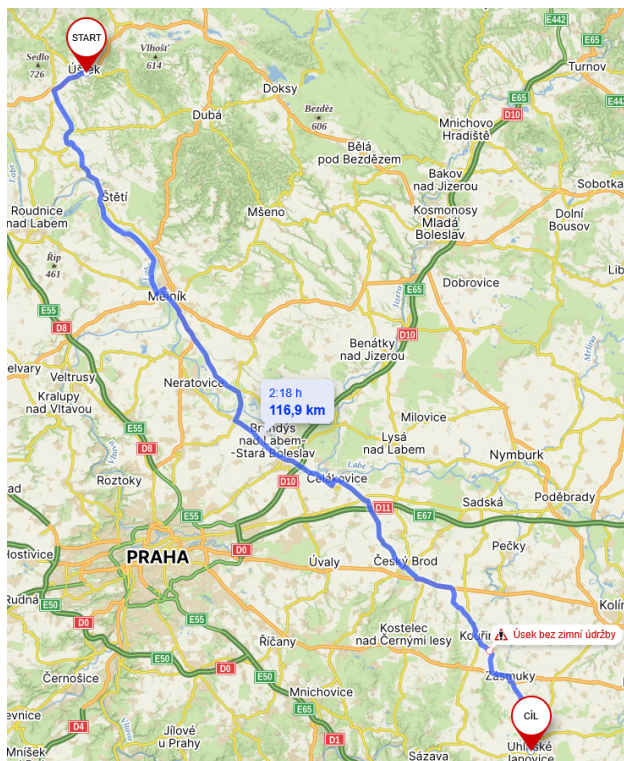
Obrázek 15: Nejkratší cesta ze stránky Apple Maps mezi městy Svojšín a Kanina



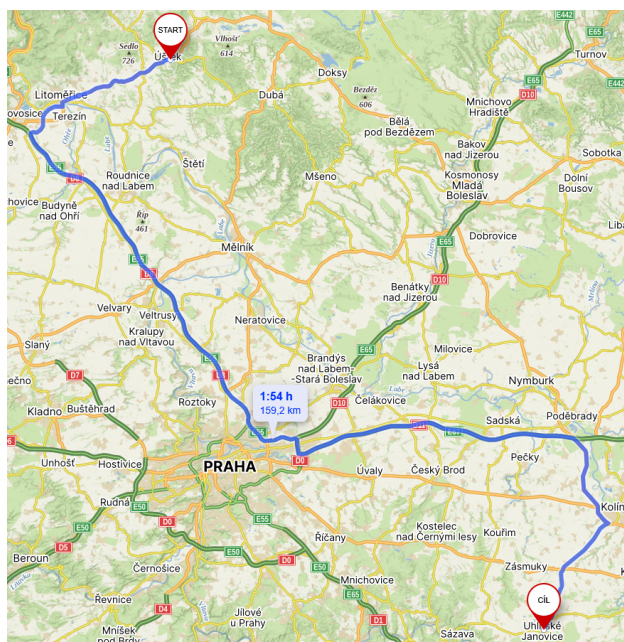
Obrázek 16: Nejrychlejší cesta ze stránky Apple Maps mezi městy Svojšín a Kanina



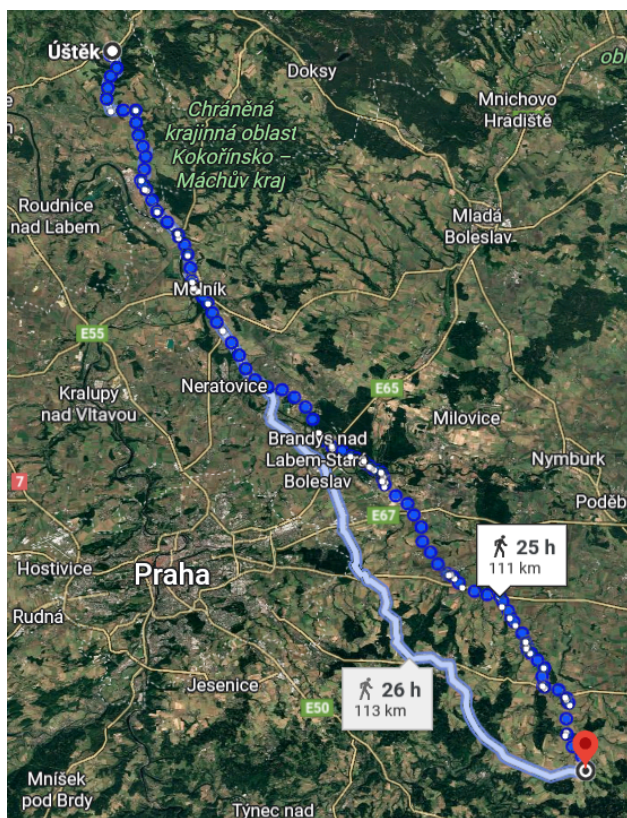
Obrázek 17: Vypočtená trasa mezi městy Ústí a Uhliřské Janovice



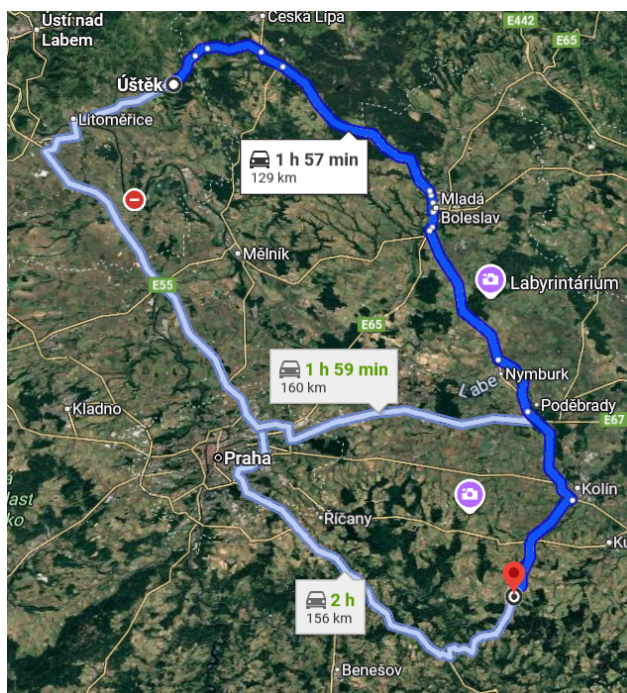
Obrázek 18: Nejkratší cesta ze stránky Mapy.com mezi městy Ústí nad Labem a Uhlířské Janovice



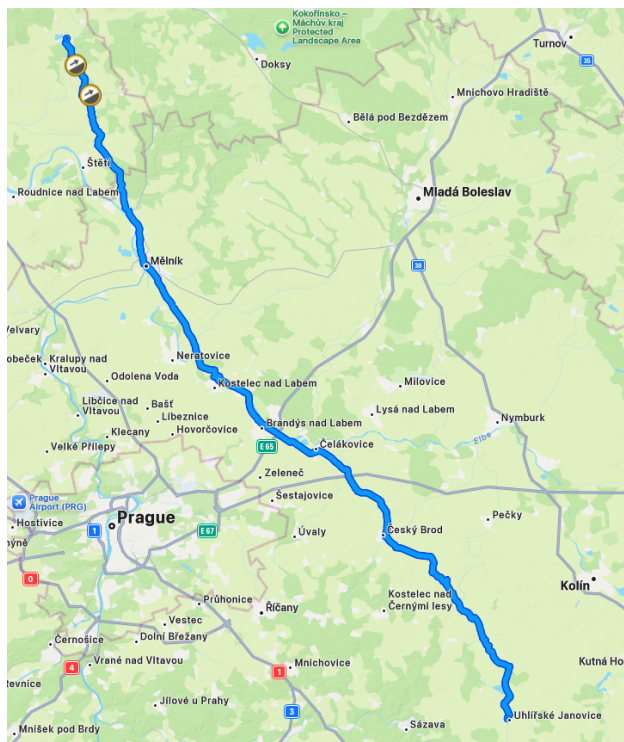
Obrázek 19: Nejrychlejší cesta ze stránky Mapy.com mezi městy Ústí nad Labem a Uhlířské Janovice



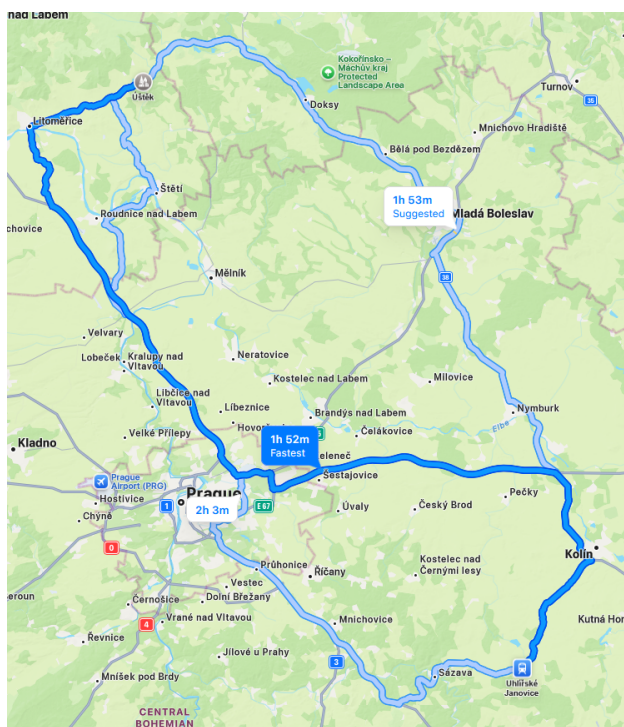
Obrázek 20: Nejkratší cesta ze stránky Google Maps mezi městy Ústě a Uhlířské Janovice



Obrázek 21: Nejrychlejší cesta ze stránky Google Maps mezi městy Ústě a Uhlířské Janovice



Obrázek 22: Nejkratší cesta ze stránky Apple Maps mezi městy Ústěk a Uhlířské Janovice



Obrázek 23: Nejrychlejší cesta ze stránky Apple Maps mezi městy Ústěk a Uhlířské Janovice

Závěr

Veškeré povinné a bonusové úlohy byly v této úloze splněny.

Pro nejkratší cestu byly implementovány algoritmy Dijkstra pro graf o nezáporných hranách a Bellman-Ford pro grafy obsahující záporné hrany. Pro grafy o nezáporných hranách tyto algoritmy pořízují shodné výsledky s tím, že Dijkstra je pro tento případ efektivnější.

Pro volbu mezi těmito algoritmy byla vytvořena funkce `shortest_path()`, která zjistí informaci, zda v grafu existují záporné hrany, a zvolí patřičný algoritmus. Zároveň při nezádaní startovní uzlu tato funkce provede výpočet nejkratší cesty mezi všemi dvojicemi uzlů, která pro tento případ trvá kolem 8 minut. Tento výpočet je volaný na konci skriptu (příloha 1) a je doplněn o psaní dosavadního postupu pro ujistění uživatele, že výpočet probíhá a nenastalo k nekonečnému cyklu.

Pro minimální kostru byly implementovány oba algoritmy Kruskal a Prime. Algoritmy pořízují, až na výjimky, stejné výsledky. Jediná změna nastává, když by nebyl graf souvislý, jelikož algoritmus Kruskal izolované hrany do kostry připojí, avšak algoritmus Prime ne. V hlavním skriptu byl použit algoritmus Kruskal, jelikož je pro tato data výrazně efektivnější.

Pro algoritmus Kruskal byly implementovány i bonusové heuristiky, které urychlují samotný výpočet.

Vypočtené Eukleidovské vzdálenosti jsou srovnatelné s výsledky z dalších navigačních aplikací. Větší rozdíly nastávají při nejkratším transportním čase, což je způsobeno vstupními daty, které nejsou doplněny případnými omezeními, které se na českých silnicích ve skutečnosti vyskytují. Podle našich výpočtů by se uživatel dostal do cílové destinace přibližně o 15 % celkového času rychleji.

Navzdory jistému zjednodušení vstupních dat, se vypočítané cesty často shodují či přibližují k těm z navigačních aplikací. Také bylo překvapivé, že se trasy v různých mapových aplikacích mohou výrazně lišit, pravděpodobně kvůli implementaci různých typů metrik a parametrů pro výpočet vlivu provozu.

Přílohy

- Příloha 1 – hlavní spouštěcí skript `U3_grafy.py`
- Příloha 2 – skript `file_to_graph_algorithms.py` obsahující funkce pro zpracování dat
- Příloha 3 – skript `graph_algorithms.py` obsahující grafové funkce
- Příloha 4 – skript `plot_algorithms.py` obsahující vykreslovací funkce
- Příloha 5 – polygonová vrstva krajů `kraje.shp`
- Příloha 6 – polygonová vrstva okresů `okresy.shp`
- Přílohy 7-9 – vstupní data silnic s cenami jednotlivých hran (`s1_euklid.txt`, `s2_rychlost.txt`, `s3_rychlost_klikatost`)
- Příloha 10 – liniová vrstva použitých silnic `silnice_final_final.shp`
- Příloha 11 – souřadnice obcí `obce_souradnice.txt`

Reference

- [1] DEMEL, Jiří. Grafy a jejich aplikace. Vyd. 2., (Vlastním nákladem 1.). Libčice nad Vltavou: J. Demel, 2015. ISBN ISBN 978-80-260-7684-1.
- [2] Výklad ze cvičení – doc. Ing. Tomáš Bayer, Ph.D.
- [3] 155YGEI Geoinformatika. Online. 2025. Dostupné z: https://geo.fsv.cvut.cz/gwiki/155YGEI_Geoinformatika. [cit. 2025-12-02].

Podepsáno dne 2. 12. 2025 v Praze

Králič Adam, Matějková Barbora